

1. Algoritmo empleado en la solución

Como tal, lo que se llevó a cabo fue un recorrido de estados que lo que hace es captar simultáneamente la posición de Samus en el laberinto, y la energía disponible en cada paso. Cada paso se marca como una tupla (p, e) , en donde “p” hace alusión a la plataforma en donde está parado actualmente, y “e” refleja la energía restante, que teniendo en consideración el enunciado, disminuye al teletransportarse.

Al representar la solución como un grafo implícito cuyos nodos son dichos estados, lo que pretendimos llevar a cabo fue un recorrido BFS, que explora todas las transiciones posibles hasta poder alcanzar la plataforma final. Ahora bien, cabe destacar que las acciones permitidas en cada estado responden a los tres mecanismos que Samus puede usar para moverse:

- a. Avanzar o dar un paso a la derecha (“C+”) o hacia la izquierda (“C-”), que no consume su energía, pero si cambia la posición en la que se encuentra.
- b. Salto especial, que refiere al momento en el que la plataforma posee un poder de salto de longitud K . Con este Samus puede impulsarse hasta $p + k$ (“S+”), o devolverse $p - k$ (“S-”), sin emplear ninguna energía.
- c. Teletransporte, que es usado para saltar varias casillas en un solo movimiento, y que tiene un costo de energía $q - p$. Esta acción es denominada como “T+(d)” en el avance, o “T-(d)” al devolverse.

En la construcción realizada, las plataformas que tienen robots quedan excluidas del grafo, esto fue considerado debido a que no tiene sentido tener viva la posibilidad de desplazarse a alguna casilla con alguno de ellos. Por esa razón es que el BFS funciona a la perfección, ya que únicamente considera caminos que estén disponibles o que sean correctos, optimizando el número de movimientos.

SOBRE EL USO DE BFS

Sobre BFS, es posible decir que cada tipo de acción que se mencionó previamente cuenta como un solo paso, por lo que recorrer el grafo en niveles asegura encontrar la solución con la mínima cantidad de movimientos. Conceptualmente, la forma en la que opera este algoritmo es la siguiente:

Primero se inicializa la cola y se agrega el estado inicial $(0, E_0)$, donde E_0 es la energía inicial con la que sale Samus. Se marca ese nodo como visitado para no volver a procesarlo, pues no tendría sentido.

Posteriormente se saca la cola el nodo o estado (p, e) . Si $p = n$, se ha alcanzado el final de grafo y por ende se detiene la búsqueda. En caso de no ser así, se verifica a los vecinos explorando las tres posibilidades de movimientos que fueron descritos anteriormente,

siempre revisando que cada nuevo nodo no haya sido visitado, y que igualmente cumplan con las condiciones de energía y de no poseer robots.

Finalmente, junto con la marca de nodo visitado, se almacena cada nodo desde el cuál llegó anteriormente (padre), ya que luego permitiría hacer Backtracking o reconstruir la secuencia de pasos que se siguió hasta finalizar la iteración.

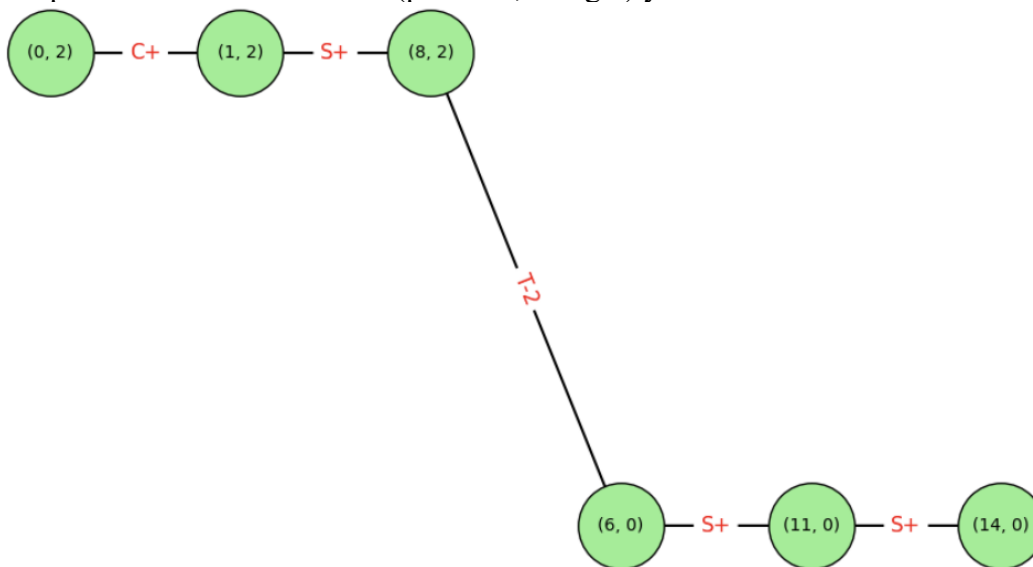
Habiendo expuesto las razones por las cuales implementamos BFS, cabe destacar un hecho importante y son las razones por las que no se emplearon otros algoritmos. Inicialmente, hay que indicar que Dijkstra no habría funcionado puesto que nuestro grafo tiene aristas de pesos iguales o es no ponderado (ya que cada paso se cuenta como 1). Por otro lado, usar DP no habría sencillo de implementar y pensar debido a los diferentes saltos y teletransportaciones hechos, a pesar de haber sido una buena idea para el Backtracking.

2. Gráfica de Ejemplo

Caso de prueba ($n = 14$, $e = 2$):

Plataformas seguras: 0,1,2,3,6,8,11,11

Caso optimo: Cada nodo muestra (posición, energía) y las flechas indican las acciones.



1. **Estado Inicial (0, 2):** Samus comienza en la plataforma 0 con 2 unidades de energía.
2. **Acción 1 - Movimiento Normal (C+):**
 - a. Avanza a la plataforma 1.
 - b. **Energía:** 2 (no se consume).
3. **Acción 2 - Salto con Poder (S+):**
 - a. Usa el poder de la plataforma 1 para saltar 7 plataformas hacia adelante (1 → 8).
 - b. **Energía:** 2 (no se consume).
4. **Acción 3 - Teletransporte (T-2):**

- a. Se teletransporta a la plataforma 6 ($8 \rightarrow 6$).
- b. **Costo de energía:** 2 unidades (distancia recorrida: 2 plataformas).
- c. **Energía restante:** 0.
- 5. **Acción 4 - Salto con Poder (S+):**
 - a. Usa el poder de la plataforma 6 para saltar 5 plataformas hacia adelante ($6 \rightarrow 11$).
 - b. **Energía:** 0 (ya agotada, pero los saltos no la consumen).
- 6. **Acción 5 - Salto con Poder (S+):**
 - a. Usa el poder de la plataforma 11 para saltar 3 plataformas hacia adelante ($11 \rightarrow 14$).
 - b. **Resultado:** Samus llega a la plataforma final.

3. Análisis de complejidades

- Complejidad Temporal:

Complejidad Proyecto 2		
Operación	Descripción	Complejidad
Inicialización (C_1)	Creación de estructuras de datos iniciales	$O(n * E)$
Operaciones en BFS (C_2)	Procesamiento de cada nodo en la cola	$O(1)$ por operación
Movimientos normales (C_3)	Evaluación de movimientos C+ y C-	$O(1)$ por movimiento
Salto especiales (C_4)	Evaluación de movimientos S+ y S-	$O(1)$ por salto
Teletransporte (C_5)	Evaluación de teletransportes posibles	$O(\log n)$ por ordenamiento
Reconstrucción (C_6)	Reconstrucción del camino solución	$O(n)$

De este modo, es posible decir que la complejidad asintótica del código realizado es de $O(n * E)$.

- Complejidad espacial: En el peor de los casos, también se usaría $O(n * E)$ de espacio en memoria, ya que con plataformas desde 0 hasta n y con niveles de energía de 0 a E, la matriz de visitación visitado ocupa memoria proporcional a $n * E$. Esto significa que, en el peor de los casos, se hace uso de una complejidad espacial de $O(n * E)$.